

Comparación de Algoritmos de Ordenamiento Bajo Condiciones de Error

Problema por desarrollar: Imagina que estás a cargo del software de navegación de una nave espacial que debe pasar cerca del Sol para una misión de recolección de datos. Debido a la intensa radiación solar, los sistemas electrónicos de la nave son susceptibles a fallos temporales, especialmente en la unidad de procesamiento encargada de la toma de decisiones críticas.

Uno de los sistemas afectados es el que organiza las señales y datos recibidos de los sensores de la nave, un proceso que requiere algoritmos de ordenamiento. Sin embargo, debido a la radiación, existe una probabilidad de error (desconocida) en las comparaciones de los algoritmos, lo que puede llevar a resultados incorrectos.

En este escenario, es crucial que el algoritmo de ordenamiento sea robusto a errores de comparación para evitar fallos en la operación de la nave.

Objetivo: El objetivo es que el algoritmo ordene los datos lo mejor posible (el ordenamiento perfecto no es necesario pero mientras mejor, mejor).

Elaboración: Comparar el desempeño de diferentes algoritmos de ordenamiento (como QuickSort, Merge Sort, Bubble Sort, Insertion Sort y Selection Sort) bajo condiciones en las que hay una probabilidad de error en las comparaciones. Considera diferentes probabilidades de error, define cómo calificar que tan ordenado está un arreglo y establece cuál de los algoritmos es el mejor para diferentes probabilidades de error.

Desarrollo:

Por lo anterior, sabemos que cada vez que compramos dos elementos existe una probabilidad de error a la que le asignaremos la letra p , la cual cambia el resultado de la comparación de manera aleatoria. El error lo vamos a manejar de la siguiente manera:

- Vamos a partir la probabilidad en dos partes: la parte de error que vamos a obtener en cada uso y la parte de NO error. Por lo tanto, una sería la probabilidad (p) de error y la probabilidad ($1-p$) de NO error.

Dado que en cada escenario que utilicemos este algoritmo para ordenar el arreglo de manera correcta, ocurrirá un error entonces necesitamos un método que nos diga que tan bien está ordenado este arreglo.

Un método posible sería definir un índice de ordenamiento parcial, el cual, una vez terminado el ordenamiento empezará a ver qué tan bien está nuestro algoritmo, sería

como partir al arreglo en subarreglos y que tenga un contador parcial para ir evaluando que tan bien está. Esta métrica como hablamos de probabilidades se moverá entre 0 y 1. El 0 como completamente desordenado y 1 como completamente ordenado.

Pasos por seguir en la prueba:

- 1.- Crear un arreglo desordenado.
- 2.- Aplicar cada algoritmo de ordenamiento teniendo en cuenta que existe la presencia de errores de comparación.
- 3.- Obtener la métrica, es decir nuestro contador parcial para poder evaluar nuestro algoritmo.

Para cada valor de p (probabilidad de error), compararemos los resultados de los algoritmos de ordenamiento usando nuestra métrica y así determinaremos cuál algoritmo es más robusto bajo diferentes niveles de error.

Nota: countInversions es la métrica especificada más arriba para medir que tan desordenado está el arreglo y compareWithError para simular los errores en las comparaciones.

Código documentado:

```
import java.util.Random;

public class SortingWithErrors {

    private static final Random rand = new Random();

    // Método para comparar dos elementos con una probabilidad de error
    public static int compareWithError(int x, int y, double pError) {
        if (rand.nextDouble() < pError) {
            return Integer.compare(y, x); // Devolvemos el resultado incorrecto
        }
        return Integer.compare(x, y); // Devolvemos el resultado correcto
    }

    // Bubble Sort
    public static void bubbleSort(int[] arr, double pError) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (compareWithError(arr[j], arr[j + 1], pError) > 0) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
}
```

```
}

// Insertion Sort
public static void insertionSort(int[] arr, double pError) {
    for (int i = 1; i < arr.length; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && compareWithError(arr[j], key, pError) > 0) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

// Selection Sort
public static void selectionSort(int[] arr, double pError) {
    for (int i = 0; i < arr.length - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < arr.length; j++) {
            if (compareWithError(arr[minIdx], arr[j], pError) > 0) {
                minIdx = j;
            }
        }
        int temp = arr[minIdx];
        arr[minIdx] = arr[i];
        arr[i] = temp;
    }
}

// Merge Sort
public static void mergeSort(int[] arr, double pError) {
    if (arr.length > 1) {
        int mid = arr.length / 2;
        int[] left = new int[mid];
        int[] right = new int[arr.length - mid];

        System.arraycopy(arr, 0, left, 0, mid);
        System.arraycopy(arr, mid, right, 0, arr.length - mid);

        mergeSort(left, pError);
        mergeSort(right, pError);
        merge(arr, left, right, pError);
    }
}

public static void merge(int[] arr, int[] left, int[] right, double pError) {
    int i = 0, j = 0, k = 0;
    while (i < left.length && j < right.length) {
        if (compareWithError(left[i], right[j], pError) <= 0) {
            arr[k++] = left[i++];
        } else {
            arr[k++] = right[j++];
        }
    }
}
```

```
    }
}
while (i < left.length) {
    arr[k++] = left[i++];
}
while (j < right.length) {
    arr[k++] = right[j++];
}
}

// Quick Sort
public static void quickSort(int[] arr, int low, int high, double pError) {
    if (low < high) {
        int pi = partition(arr, low, high, pError);
        quickSort(arr, low, pi - 1, pError);
        quickSort(arr, pi + 1, high, pError);
    }
}

public static int partition(int[] arr, int low, int high, double pError) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (compareWithError(arr[j], pivot, pError) <= 0) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return i + 1;
}

// Método para contar el número de inversiones
public static int countInversions(int[] arr) {
    int invCount = 0;
    for (int i = 0; i < arr.length - 1; i++) {
        for (int j = i + 1; j < arr.length; j++) {
            if (arr[i] > arr[j]) {
                invCount++;
            }
        }
    }
    return invCount;
}
```